

Contents

Chapter 1: Image Warping and Stitching	2
1.1 Image Warping	2
1.2 Interpolation for Inverse Warping	2
1.3 Panoramas and Image Stitching	2
Chapter 2: The Pinhole Camera Model	4
2.1 Physical Setup and Coordinate Systems	4
2.2 Perspective Projection	4
2.3 Camera Matrix and Homogeneous Coordinates	4
2.4 Intrinsic Matrix K	4
2.5 Extrinsic Parameters	5
2.6 Full Camera Matrix	5
2.7 Perspective Distortion	5
2.8 Alternative Camera Models	5
2.9 Pose Estimation and Camera Calibration	5
2.10 Camera Matrix Decomposition	6
2.11 Nonlinear Refinement and Lens Distortion	6
2.12 Calibration Methods	6
2.13 Augmented Reality Application	6

Chapter 1: Image Warping and Stitching

1.1 Image Warping

Applying a geometric transformation to an image requires deciding how to compute pixel values at non-integer locations.

Forward warping. For each input pixel (x, y) , compute the output location $\mathbf{x}' = T(\mathbf{x})$ and write the value there.

- **Problem:** output pixels can be *missed* (holes) or *overwritten* (conflicts) because the map from input to output is not one-to-one on the integer grid.
- Rarely used in practice.

Inverse warping (preferred). For each output pixel $\mathbf{x}'$, compute the source location $\mathbf{x} = T^{-1}(\mathbf{x}')$ in the input and interpolate:

$$g(\mathbf{x}') = f(T^{-1}(\mathbf{x}')).$$

Every output pixel is filled; no holes. Requires T^{-1} to be available (always the case for the five transforms above).

1.2 Interpolation for Inverse Warping

When the inverse map $T^{-1}(\mathbf{x}')$ returns a non-integer location (x, y) , we cannot directly look up a pixel value. Interpolation reconstructs a continuous signal from the discrete grid so that the value at any real-valued coordinate can be estimated.

1.2.1 Nearest-Neighbor Interpolation

The output is assigned the value of the closest grid sample:

$$f(x, y) \approx f(\lfloor x \rfloor, \lfloor y \rfloor),$$

where $\lfloor \cdot \rfloor$ denotes rounding to the nearest integer.

- **Cost:** $O(1)$ — one table lookup.
- **Continuity:** C^{-1} (piecewise constant; discontinuities at half-integer boundaries).
- **Artifacts:** Blocky/pixelated appearance, visible staircase edges on diagonal or curved features.
- **Use case:** Label maps, segmentation masks, or any application where the discrete class label must not be blended.

1.2.2 Bilinear Interpolation

Let $(x_0, y_0) = (\lfloor x \rfloor, \lfloor y \rfloor)$ be the integer floor, and define fractional offsets $\alpha = x - x_0$, $\beta = y - y_0 \in [0, 1)$. The four surrounding samples are

$$\begin{aligned} f_{00} &= f(x_0, y_0), & f_{10} &= f(x_0+1, y_0), \\ f_{01} &= f(x_0, y_0+1), & f_{11} &= f(x_0+1, y_0+1). \end{aligned}$$

The bilinear estimate is obtained by first interpolating along rows, then along the column:

$$\begin{aligned} f(x, y_0) &= (1 - \alpha) f_{00} + \alpha f_{10}, \\ f(x, y_0+1) &= (1 - \alpha) f_{01} + \alpha f_{11}, \\ f(x, y) &= (1 - \beta) f(x, y_0) + \beta f(x, y_0+1). \end{aligned}$$

Expanding, the compact form is

$$f(x, y) \approx (1 - \alpha)(1 - \beta) f_{00} + \alpha(1 - \beta) f_{10} + (1 - \alpha)\beta f_{01} + \alpha\beta f_{11}.$$

Equivalently, in matrix form:

$$f(x, y) \approx \begin{bmatrix} 1 - \alpha & \alpha \end{bmatrix} \begin{bmatrix} f_{00} & f_{01} \\ f_{10} & f_{11} \end{bmatrix} \begin{bmatrix} 1 - \beta \\ \beta \end{bmatrix}.$$

- **Cost:** 3 lerps (or 4 multiplications + 3 additions per channel).
- **Continuity:** C^0 — the surface is continuous but its partial derivatives are discontinuous at integer boundaries.
- **Artifacts:** Mild blurring; no blocky artifacts. High-frequency detail is attenuated because the interpolating kernel (tent function) has a non-ideal frequency response.
- **Use case:** Default choice for image warping, texture mapping, and image resizing in real-time pipelines.

1.2.3 Bicubic Interpolation

Bicubic interpolation fits a degree-3 polynomial surface through a 4×4 neighborhood of 16 samples. The standard Keys cubic kernel with parameter $a = -0.5$ is

$$w(t) = \begin{cases} (a+2)|t|^3 - (a+3)|t|^2 + 1 & 0 \leq |t| < 1, \\ a|t|^3 - 5a|t|^2 + 8a|t| - 4a & 1 \leq |t| < 2, \\ 0 & |t| \geq 2. \end{cases}$$

The interpolated value is

$$f(x, y) \approx \sum_{m=-1}^2 \sum_{n=-1}^2 f(x_0+m, y_0+n) w(m-\alpha) w(n-\beta).$$

- **Cost:** 16 samples; separable — compute four cubic 1-D interpolations along x , then one along y (12 multiplications per pass).
- **Continuity:** C^1 — both the function and its first derivatives are continuous.
- **Artifacts:** Slight ringing near sharp edges (Gibbs-like overshoot) due to the negative lobes of the kernel.
- **Use case:** High-quality image scaling, photo editing software, and medical imaging.

1.2.4 Sinc / Lanczos Interpolation

The ideal reconstruction filter for a band-limited signal is the sinc:

$$f(x, y) = \sum_m \sum_n f(m, n) \text{sinc}(x - m) \text{sinc}(y - n), \quad \text{sinc}(t) = \frac{\sin(\pi t)}{\pi t}.$$

Because the sinc has infinite support it is truncated and windowed. The Lanczos- a kernel multiplies sinc by a sinc-shaped window of radius a (typically $a = 2$ or 3):

$$L_a(t) = \begin{cases} \text{sinc}(t) \text{sinc}(t/a) & |t| < a, \\ 0 & |t| \geq a. \end{cases}$$

- **Cost:** $(2a)^2$ samples; $a = 3$ requires 36 samples per output pixel.
- **Continuity:** C^0 at window boundary; approximately C^∞ elsewhere.
- **Artifacts:** Best preservation of fine detail and frequency content; some ringing possible for $a = 2$.
- **Use case:** Offline high-quality downsampling and super-resolution pre-processing.

1.2.5 Comparison and Practical Guidelines

Method	Support	Continuity	Sharpness	Speed
Nearest-neighbor	1×1	C^{-1}	Highest (aliased)	Fastest
Bilinear	2×2	C^0	Medium	Fast
Bicubic	4×4	C^1	High	Moderate
Lanczos-3	6×6	$\approx C^\infty$	Highest (clean)	Slow

Key design decisions:

- **Upsampling** (output larger than input): bicubic or Lanczos avoids the blurring of bilinear.
- **Downsampling** (output smaller than input): always *anti-alias first* by low-pass filtering before sampling; otherwise all methods alias.
- **Discrete labels** (segmentation, depth maps): nearest-neighbor is mandatory to prevent blending across class boundaries.
- **Real-time constraints:** bilinear is hardware-accelerated in every GPU texture unit and is the pragmatic default.

1.3 Panoramas and Image Stitching



Figure 1: Alcatraz Island, shown in a panorama created by image stitching

Overview. Panorama creation stitches overlapping photographs captured from (approximately) the same optical centre into a single wide-angle composite. Standard pipeline: **(1)** detect and match features across image pairs; **(2)** robustly estimate a homography per pair via RANSAC+DLT; **(3)** project all images onto a common surface (plane, cylinder, or sphere); **(4)** warp and composite; **(5)** blend seams. A homography relates two images whenever the camera undergoes a *pure rotation* (fixed optical centre) or the scene is *planar*.

Homography Estimation - Direct Linear Transform (DLT). Given $n \geq 4$ point correspondences $\{\mathbf{x}_i \leftrightarrow \mathbf{x}'_i\}$, DLT finds H by forming a homogeneous linear system. Since $\tilde{\mathbf{x}}'_i \sim H\tilde{\mathbf{x}}_i$ (homogeneous equality), the cross-product condition

$$\tilde{\mathbf{x}}'_i \times H\tilde{\mathbf{x}}_i = \mathbf{0}$$

expands (for $\tilde{\mathbf{x}}_i = (x_i, y_i, 1)^T$ and $\tilde{\mathbf{x}}'_i = (x'_i, y'_i, 1)^T$) to two independent equations:

$$A_i \mathbf{h} = \mathbf{0}, \quad A_i = \begin{bmatrix} \mathbf{0}^T & -\tilde{\mathbf{x}}_i^T & y'_i \tilde{\mathbf{x}}_i^T \\ \tilde{\mathbf{x}}_i^T & \mathbf{0}^T & -x'_i \tilde{\mathbf{x}}_i^T \end{bmatrix} \in \mathbb{R}^{2 \times 9},$$

where $\mathbf{h} = (h_{11}, h_{12}, \dots, h_{33})^T \in \mathbb{R}^9$. Stacking all n correspondences:

$$A\mathbf{h} = \mathbf{0}, \quad A \in \mathbb{R}^{2n \times 9}.$$

The solution is the *right singular vector* of A for the *smallest singular value* (via SVD), minimizing $\|A\mathbf{h}\|$ subject to $\|\mathbf{h}\| = 1$. With $n = 4$ correspondences A is 8×9 , the null space is one-dimensional, and H is determined up to scale (8 DOF).

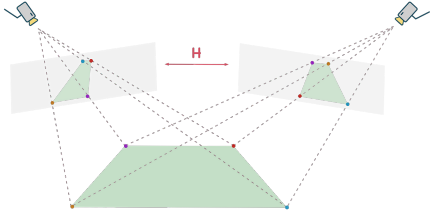


Figure 2: Homography Estimation

Normalization (essential). Raw pixel coordinates (spanning hundreds to thousands of units) make A severely ill-conditioned. Before forming A , independently normalize each point set:

1. Translate so the centroid is at the origin:

$$\bar{\mathbf{x}}_i = \mathbf{x}_i - \boldsymbol{\mu}$$

2. Scale uniformly so the mean distance from the origin is $\sqrt{2}$:

$$s = \sqrt{2}/\|\bar{\mathbf{x}}_i\|$$

3. The normalisation matrix is

$$T = \begin{bmatrix} s & 0 & -s\mu_x \\ 0 & s & -s\mu_y \\ 0 & 0 & 1 \end{bmatrix}$$

Run DLT on $\hat{\mathbf{x}}_i = T\mathbf{x}_i$ and $\hat{\mathbf{x}}'_i = T'\mathbf{x}'_i$ to obtain $\hat{H}$; then denormalise: $H = T'^{-1}\hat{H}T$. Normalisation reduces condition numbers by orders of magnitude and is mandatory for numerical reliability (Hartley, 1997).

RANSAC for robust estimation. Descriptor matching yields many outliers. RANSAC applies with minimum sample $s = 4$ (to fit H) and inlier threshold $\epsilon \approx 3$ pixels on reprojection error $\|\mathbf{x}'_i - H\mathbf{x}_i\|$. After RANSAC, refit H on all inliers via DLT. Required iterations: $k = \log(1-p)/\log(1-(1-e)^4)$; for $e = 0.5$, $p = 0.99$: $k \approx 72$.

Non-linear refinement. DLT minimises an *algebraic* error with no geometric meaning. For higher accuracy, refine H by minimising the **symmetric reprojection error**:

$$\min_H \sum_i \left(d(\mathbf{x}'_i, H\mathbf{x}_i)^2 + d(\mathbf{x}_i, H^{-1}\mathbf{x}'_i)^2 \right),$$

solved by Levenberg–Marquardt initialised from the DLT solution.

Cylindrical projection. For wide-angle panoramas (> 60 field of view), projective stitching produces severe distortion near the periphery. Instead, project each image onto a *cylinder* of radius f (focal length in pixels). For a point (x, y) with origin at the image center:

$$\hat{x} = f \arctan\left(\frac{x}{f}\right), \quad \hat{y} = \frac{fy}{\sqrt{x^2 + f^2}}.$$

In cylindrical coordinates, a pure camera rotation about the vertical axis becomes a *horizontal translation*, making alignment trivial for rotation-only sequences. The focal length f must be known (from calibration or EXIF).

Spherical projection. For full 360×180 panoramas, project onto a sphere of radius f . Spherical angles of image point (x, y) :

$$\phi = \arctan\left(\frac{x}{f}\right), \quad \lambda = \arctan\left(\frac{y}{\sqrt{x^2 + f^2}}\right).$$

Re-project to *equirectangular* (longitude–latitude) format: $(\phi, \lambda) \rightarrow (f\phi, f\lambda)$. Used in 360° video, VR headsets, and Google Street View.

Image blending. Warped images must be composited; seams appear due to exposure differences, vignetting, and sub-pixel misalignment.

Alpha feathering. Assign each pixel a weight $w(\mathbf{x})$ proportional to its distance from the image boundary. In overlap regions blend linearly:

$$C(\mathbf{x}) = \frac{w_A(\mathbf{x})C_A(\mathbf{x}) + w_B(\mathbf{x})C_B(\mathbf{x})}{w_A(\mathbf{x}) + w_B(\mathbf{x})}.$$

Simple to implement; can produce ghosting under misalignment.

Multiband blending (Burt & Adelson, 1983). Low-frequency content should be blended with a wide transition (to hide color shifts); high-frequency content should be cut sharply (to preserve edge sharpness). Algorithm:

1. Build Gaussian pyramids $G_A^{(\ell)}, G_B^{(\ell)}$ and mask pyramid $G_M^{(\ell)}$.
2. Build Laplacian pyramids: $L_A^{(\ell)} = G_A^{(\ell)} - \text{up}(G_A^{(\ell+1)})$, similarly $L_B^{(\ell)}$.
3. Blend each level: $L_S^{(\ell)} = G_M^{(\ell)} \cdot L_A^{(\ell)} + (1 - G_M^{(\ell)}) \cdot L_B^{(\ell)}$.
4. Collapse the blended Laplacian pyramid to reconstruct the composite.

Seam finding (graph cut). Rather than blending the entire overlap, find a *minimum-cost cut* through the overlap where the two images agree most closely (minimum photometric difference). Kwatra et al. (2003) formulate this as a Markov Random Field and solve with graph cuts, eliminating ghosting entirely.

Bundle adjustment for multi-image panoramas. With $m > 2$ images, composing pairwise homographies accumulates drift. Global alignment parameterises each camera by its rotation $R_i \in SO(3)$ and focal length f_i , then jointly minimises total reprojection error over all matched image pairs (i, j) and inlier correspondences k :

$$\min_{\{R_i, f_i\}} \sum_{(i,j) \in \mathcal{E}} \sum_{k \in \mathcal{M}_{ij}} \left\| \mathbf{x}_k^{(j)} - \pi\left(R_j R_i^{-1} \pi^{-1}(\mathbf{x}_k^{(i)}; f_i); f_j\right) \right\|^2,$$

where $\pi(\cdot; f)$ is projection with focal length f and π^{-1} back-projects to 3D. Solved by Levenberg–Marquardt (AutoStitch, Brown & Lowe, 2007). **Gain compensation** (per-image exposure scale g_i) is typically added to the objective to handle photometric inconsistency between images.

Chapter 2: The Pinhole Camera Model

A camera is a device that maps the 3-D world onto a 2-D image sensor. The **pinhole camera** is the simplest mathematical model of this process: all light rays pass through a single point (the *pinhole*) before hitting the image plane. Despite its simplicity, this model underlies virtually all modern computer vision geometry.

The complete mapping from a 3-D world point $\mathbf{X}_w$ to a 2-D image point $\mathbf{x}$ is written compactly as

$$\mathbf{x} \sim P \mathbf{X}_w,$$

where P is a 3×4 **camera matrix** and “ $\sim$ ” denotes equality up to a non-zero scale (homogeneous equality).

2.1 Physical Setup and Coordinate Systems

2.1.1 Physical components

- **Aperture (pinhole):** the single point through which all light rays must pass. Making it infinitely small gives a perfectly sharp image but admits no light; real cameras use a finite aperture and a lens to focus.
- **Image plane:** the sensor surface at distance f behind the pinhole where the image forms. Each scene point traces a straight ray through the pinhole and lands on the image plane.
- **Camera center (optical center / center of projection):** the pinhole location itself; the geometric origin of the model. All projection rays emanate from here.
- **Focal length f :** the distance from the pinhole to the image plane. A larger f magnifies the image (telephoto); a smaller f widens the field of view (wide-angle).
- **Principal axis (optical axis):** the ray through the camera center perpendicular to the image plane.
- **Principal point (p_x, p_y) :** where the principal axis intersects the image plane. Ideally the image center, but shifts due to manufacturing imperfections.

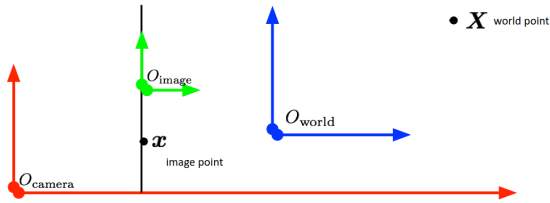
2.1.2 Three coordinate frames

Three reference frames are needed to describe the full projection pipeline:

- **World frame $(\tilde{\mathbf{X}}_w)$:** a fixed global frame in which 3-D scene points are defined. Arbitrary choice.
- **Camera frame $(\tilde{\mathbf{X}}_c)$:** origin at the camera center, z -axis along the principal axis pointing into the scene. The natural frame for projection.
- **Image frame (u, v) :** 2-D pixel coordinates on the sensor, with origin at the image corner (not the principal point).

The full pipeline is:

$$\underbrace{\tilde{\mathbf{X}}_w}_{\text{world}} \xrightarrow[\text{extrinsics}]{[\mathbf{R}|\mathbf{t}]} \underbrace{\tilde{\mathbf{X}}_c}_{\text{camera}} \xrightarrow[\text{projection}]{\text{divide by } z} \underbrace{(x_n, y_n)}_{\text{normalized}} \xrightarrow[\text{intrinsics}]{K} \underbrace{(u, v)}_{\text{pixel}}.$$



2.2 Perspective Projection

2.2.1 Derivation from similar triangles

Place the camera center at the origin with the image plane at $z = f$. A 3-D point in camera coordinates $\tilde{\mathbf{X}}_c = (X, Y, Z)^\top$ emits a ray through the origin. That ray intersects the image plane at the 2-D point (x, y) .

By similar triangles along both axes:

$$\frac{x}{f} = \frac{X}{Z} \implies x = \frac{fX}{Z}, \quad \frac{y}{f} = \frac{Y}{Z} \implies y = \frac{fY}{Z}.$$

This is the **perspective projection** (also called *central projection*):

$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \mapsto \begin{pmatrix} fX/Z \\ fY/Z \end{pmatrix}. \quad (1)$$

Two key consequences follow immediately:

- **Depth-dependent scaling:** farther objects appear smaller (magnification $m = f/Z$ decreases with Z).
- **Nonlinearity:** the division by Z makes perspective projection nonlinear in Euclidean coordinates. Homogeneous coordinates linearize it.

2.2.2 Normalized image coordinates ($f = 1$)

Setting $f = 1$ and placing the image plane at $z = 1$ gives the *normalized* (or *canonical*) projection:

$$(X, Y, Z)^\top \mapsto (X/Z, Y/Z)^\top. \quad (2)$$

These **normalized image coordinates** $(x_n, y_n) = (X/Z, Y/Z)$ are independent of any camera-specific parameters and form the intermediate representation before applying the intrinsic matrix K .

2.3 Camera Matrix and Homogeneous Coordinates

2.3.1 Why homogeneous coordinates?

Perspective projection involves division by Z , which is nonlinear. **Homogeneous coordinates** represent a 2-D point (u, v) as a 3-vector $(u, v, 1)^\top$ (or any non-zero scalar multiple thereof), and a 3-D point (X, Y, Z) as a 4-vector $(X, Y, Z, 1)^\top$. This trick turns perspective projection into a *linear* matrix multiplication: the division by Z is deferred and recovered by dividing out the last coordinate at the end (“de-homogenization”).

2.3.2 Camera as a linear map

A camera maps homogeneous 3-D world coordinates to homogeneous 2-D image coordinates:

$$\mathbf{x} \sim P \mathbf{X}_w, \quad \mathbf{x} \in \mathbb{P}^2, \quad \mathbf{X}_w \in \mathbb{P}^3, \quad (3)$$

where P is the 3×4 **camera matrix** (11 DOF after removing the overall scale ambiguity from 12 entries). Expanded:

$$\begin{pmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{pmatrix} = \begin{pmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}, \quad (u, v) = \left(\frac{\tilde{u}}{\tilde{w}}, \frac{\tilde{v}}{\tilde{w}} \right).$$

2.3.3 Simplest projection matrix ($f = 1$, camera at origin)

When the camera center coincides with the world origin and $f = 1$:

$$P_0 = [I_{3 \times 3} \mid \mathbf{0}] = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \quad (4)$$

Applying P_0 to $(X, Y, Z, 1)^\top$ yields $(X, Y, Z)^\top$ in homogeneous image coordinates, which de-homogenizes to $(X/Z, Y/Z)^\top$, recovering the normalized projection (2).

2.3.4 General focal length f

Scaling the first two rows by f encodes the image-plane distance:

$$P_f = \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad (X, Y, Z, 1)^\top \mapsto (fX/Z, fY/Z)^\top, \quad (5)$$

recovering the full perspective equation (1).

2.4 Intrinsic Matrix K

The **intrinsic matrix** K converts normalized image coordinates (x_n, y_n) to pixel coordinates (u, v) . It bundles all camera-internal geometric parameters.

2.4.1 Principal-point offset

The image origin is typically at the sensor corner, not at the principal point. Adding offset (p_x, p_y) in pixels:

$$u = f x_n + p_x, \quad v = f y_n + p_y. \quad (6)$$

This gives the standard K for a camera with square pixels:

$$K = \begin{pmatrix} f & 0 & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{pmatrix}. \quad (7)$$

Intrinsic DOF: f (1) + p_x, p_y (2) = 3 intrinsic parameters. With 6 extrinsic DOF ($\mathbf{R}$: 3, $\mathbf{t}$: 3), the standard model has **9 DOF** total.

2.4.2 CCD camera — non-square pixels

Digital sensors may have non-square pixels, giving separate effective focal lengths f_x and f_y (in pixels) along each axis:

$$K = \begin{pmatrix} f_x & 0 & p_x \\ 0 & f_y & p_y \\ 0 & 0 & 1 \end{pmatrix}, \quad f_x = f s_x, \quad f_y = f s_y, \quad (8)$$

where s_x, s_y are the pixel densities (pixels per meter). Adding f_y as a free parameter brings the intrinsic count to 4, giving **10 DOF** total.

2.4.3 Finite projective camera — sensor skew

If the pixel axes are not exactly perpendicular (e.g., due to manufacturing imperfections), a **skew** parameter s couples the x and y directions:

$$K = \begin{pmatrix} f_x & s & p_x \\ 0 & f_y & p_y \\ 0 & 0 & 1 \end{pmatrix}, \quad s \neq 0. \quad (9)$$

In practice $s \approx 0$ for modern sensors, but the general model with all 5 intrinsic parameters free gives **11 DOF** total. This is the most general *finite* camera (infinite cameras such as orthographic are excluded).

2.4.4 Degrees-of-freedom summary

Camera model	Intrinsic params	Extra	Total DOF
Standard pinhole	f, p_x, p_y	—	9
CCD (non-square pixels)	f_x, f_y, p_x, p_y	f_y	10
Finite projective (skew)	f_x, f_y, s, p_x, p_y	s	11

Extrinsic DOF (6) = rotation (3) + translation (3) in all cases.

2.5 Extrinsic Parameters

2.5.1 World-to-camera coordinate transform

The **extrinsic parameters** describe the camera's pose (position and orientation) in the world. They transform world coordinates into camera coordinates before projection.

Let $\tilde{\mathbf{C}} \in \mathbb{R}^3$ be the camera center's position in world coordinates, and $\mathbf{R} \in SO(3)$ be the 3×3 rotation matrix whose columns are the world-frame directions of the camera's x -, y -, and z -axes.

The world-to-camera transform in Euclidean coordinates is:

$$\tilde{\mathbf{X}}_c = \mathbf{R}(\tilde{\mathbf{X}}_w - \tilde{\mathbf{C}}) = \mathbf{R} \tilde{\mathbf{X}}_w \underbrace{- \mathbf{R} \tilde{\mathbf{C}}}_{\mathbf{t}}. \quad (10)$$

Key insight: $\mathbf{t} = -\mathbf{R} \tilde{\mathbf{C}}$ is *not* the camera position but rather the position of the world origin in camera coordinates.

In homogeneous block form (4×4), this is an invertible rigid-body transform:

$$\mathbf{X}_c = \underbrace{\begin{pmatrix} \mathbf{R} & -\mathbf{R} \tilde{\mathbf{C}} \\ \mathbf{0}^\top & 1 \end{pmatrix}}_{[\mathbf{R}|\mathbf{t}]_{\text{homog.}}} \mathbf{X}_w. \quad (11)$$

2.5.2 Recovering the camera center from $\mathbf{t}$

Given the compact parameterization $P = K[\mathbf{R} | \mathbf{t}]$, the camera center in world coordinates is recovered as:

$$\tilde{\mathbf{C}} = -\mathbf{R}^\top \mathbf{t}. \quad (12)$$

2.6 Full Camera Matrix

2.6.1 Assembling the pipeline

The complete projection from world point to pixel is:

$$\mathbf{X}_c = [\mathbf{R} | \mathbf{t}] \mathbf{X}_w \quad (\text{extrinsics: world} \rightarrow \text{camera}) \quad (13)$$

$$\mathbf{x}_n = P_0 \mathbf{X}_c \quad (\text{ideal projection with } f = 1) \quad (14)$$

$$\mathbf{x} = K \mathbf{x}_n \quad (\text{intrinsics: normalized} \rightarrow \text{pixel}). \quad (15)$$

Chaining these three steps gives the canonical decomposition of P :

$$P = K [I | \mathbf{0}] \begin{pmatrix} \mathbf{R} & -\mathbf{R} \tilde{\mathbf{C}} \\ \mathbf{0}^\top & 1 \end{pmatrix} = K [\mathbf{R} | \mathbf{t}], \quad \mathbf{t} = -\mathbf{R} \tilde{\mathbf{C}}. \quad (16)$$

The left 3×3 block of P is $M = K\mathbf{R}$, the product of an upper-triangular matrix (K) and an orthogonal matrix ($\mathbf{R}$). This structure enables later decomposition via QR.

2.6.2 Block-size summary

Block	Size	Structure	Physical meaning
K	3×3	upper-triangular	intrinsic camera geometry
$\mathbf{R}$	3×3	orthogonal $SO(3)$	camera orientation in world
$\mathbf{t}$	3×1	vector	world origin in camera frame
P	3×4	general rank-3	full projection matrix

2.7 Perspective Distortion

2.7.1 Depth-dependent magnification

The image magnification at depth Z is:

$$m(Z) = \frac{f}{Z}. \quad (17)$$

Because m depends on Z , objects at different depths are scaled differently. This is **perspective distortion**: parallel lines converge to *vanishing points*, and depth changes cause apparent size changes even for rigid objects.

2.7.2 The dolly-zoom (Vertigo) effect

Doubling f while simultaneously doubling Z keeps the foreground subject at the same image size ($m = f/Z = \text{const}$), but background objects — at a different depth Z' — are now magnified by $f'/(Z') = 2f/Z'$, making them appear compressed relative to the foreground. This is the basis of the **dolly-zoom** (“Vertigo effect”) in cinematography.

2.7.3 Foreshortening and vanishing points

A set of parallel lines in the world with direction vector $\mathbf{d}$ converges in the image to a single **vanishing point** at $\mathbf{v} = K\mathbf{R}\mathbf{d}$ (in homogeneous coordinates). The horizon line is the set of all vanishing points for horizontal directions, lying at the image row corresponding to the camera's eye level.

2.8 Alternative Camera Models

2.8.1 Weak perspective (scaled orthographic)

When the scene depth variation ΔZ is small compared to the average depth Z_0 (i.e., $\Delta Z \ll Z_0$), all points are approximated as being at the same depth Z_0 . The depth-dependent magnification f/Z is then replaced by a single constant scale factor $m = f/Z_0$:

$$(X, Y, Z)^\top \mapsto (mX, mY)^\top, \quad m = \frac{f}{Z_0}, \quad (18)$$

$$P_{\text{weak}} = \begin{pmatrix} m & 0 & 0 & 0 \\ 0 & m & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (19)$$

This is valid when (1) the scene is very far from the camera, or (2) a **telecentric lens** is used (pinhole placed at the front focal point, admitting only rays parallel to the principal axis).

2.8.2 Orthographic projection

The special case of weak perspective with $m = 1$ and no origin offset gives pure **orthographic projection**: depth is entirely ignored, and world x, y coordinates map directly to image coordinates.

$$P_{\text{ortho}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (20)$$

Orthographic projection preserves parallel lines, distances, and ratios along any direction — properties lost in perspective projection. It is accurate for engineering/CAD drawings and telephoto photography of distant objects.

2.9 Pose Estimation and Camera Calibration

Goal: Given N point correspondences $\{(\mathbf{X}_i, \mathbf{x}_i)\}_{i=1}^N$ between known 3-D world points and observed 2-D image points, estimate the 3×4 camera matrix P .

2.9.1 Projection equations in inhomogeneous coordinates

Let $\mathbf{m}_j^\top$ denote the j -th row of P (a 1×4 vector). The projection $\mathbf{x}_i = P\mathbf{X}_i$ in homogeneous form de-homogenizes to:

$$u_i = \frac{\mathbf{m}_1^\top \mathbf{X}_i}{\mathbf{m}_3^\top \mathbf{X}_i}, \quad v_i = \frac{\mathbf{m}_2^\top \mathbf{X}_i}{\mathbf{m}_3^\top \mathbf{X}_i}. \quad (21)$$

These are rational (nonlinear) functions of P 's entries due to the shared denominator $\mathbf{m}_3^\top \mathbf{X}_i$.

2.9.2 Linearization: Direct Linear Transform (DLT)

Cross-multiplying (21) clears the denominator and yields two *linear* equations per correspondence:

$$\mathbf{m}_1^\top \mathbf{X}_i - u_i \mathbf{m}_3^\top \mathbf{X}_i = 0, \quad (22)$$

$$\mathbf{m}_2^\top \mathbf{X}_i - v_i \mathbf{m}_3^\top \mathbf{X}_i = 0. \quad (23)$$

Stacking these for all N correspondences into a matrix $A \in \mathbb{R}^{2N \times 12}$:

$$A\mathbf{p} = \mathbf{0}, \quad \mathbf{p} = \text{vec}(P) \in \mathbb{R}^{12}. \quad (24)$$

This is a homogeneous linear system: the camera matrix P lies in the null space of A . Since P has 11 DOF (12 entries, 1 scale), each correspondence contributes 2 constraints, so $N \geq 6$ correspondences are needed (actually $N = 6$ gives exactly 11 constraints; more gives an overdetermined system solved in the least-squares sense).

2.9.3 Solution by SVD

The minimum-norm solution (constrained to $\|\mathbf{p}\| = 1$ to avoid the trivial $\mathbf{p} = \mathbf{0}$) is:

$$\hat{\mathbf{p}} = \arg \min_{\|\mathbf{p}\|=1} \|\mathbf{A}\mathbf{p}\|^2 = \text{last column of } V \text{ in } A = U\Sigma V^\top. \tag{25}$$

Equivalently, $\hat{\mathbf{p}}$ is the eigenvector of $A^\top A$ corresponding to its *smallest* eigenvalue. This is the **DLT algorithm**.

2.10 Camera Matrix Decomposition

Given an estimated P , it is useful to recover the physical parameters K , $\mathbf{R}$, and $\tilde{\mathbf{C}}$ separately.

2.10.1 Recovering the camera center $\tilde{\mathbf{C}}$

By definition, the camera center projects to the zero vector under P :

$$P \tilde{\mathbf{C}}_h = \mathbf{0} \implies \tilde{\mathbf{C}}_h = \text{null}(P). \tag{26}$$

Numerically, $\tilde{\mathbf{C}}_h$ is the right singular vector of P corresponding to the smallest singular value (from $P = U\Sigma V^\top$, take the last column of V). De-homogenize to get $\tilde{\mathbf{C}}$.

2.10.2 Recovering intrinsics K and rotation $\mathbf{R}$

Write $P = [M \mid \mathbf{p}_4]$ where $M = K\mathbf{R}$ is the left 3×3 block. Since K is upper-triangular with positive diagonal and $\mathbf{R}$ is orthogonal, their product M has a unique factorization via **RQ decomposition** (a variant of QR):

$$M = K\mathbf{R} \xrightarrow{\text{RQ}} K \text{ (upper-triangular, positive diagonal), } \mathbf{R} \text{ (orthogonal).} \tag{27}$$

Finally, $\mathbf{t} = K^{-1}\mathbf{p}_4$, and the camera center is $\tilde{\mathbf{C}} = -\mathbf{R}^\top \mathbf{t}$.

2.11 Nonlinear Refinement and Lens Distortion

2.11.1 Reprojection error

The DLT solution minimizes an algebraic error that has no direct geometric meaning. The preferred objective is the **reprojection error** — the sum of squared Euclidean distances between observed image points and their projections under the estimated P :

$$E(K, \mathbf{R}, \mathbf{t}) = \sum_{i=1}^N \left[\left(u_i - \frac{\mathbf{m}_1^\top \mathbf{X}_i}{\mathbf{m}_3^\top \mathbf{X}_i} \right)^2 + \left(v_i - \frac{\mathbf{m}_2^\top \mathbf{X}_i}{\mathbf{m}_3^\top \mathbf{X}_i} \right)^2 \right]. \tag{28}$$

Minimized over K , $\mathbf{R}$, $\mathbf{t}$ by iterative nonlinear optimization (typically Levenberg–Marquardt), initialized from the DLT solution.

2.11.2 Radial lens distortion

Real lenses deviate from the ideal pinhole model. The dominant imperfection is **radial distortion**: image points are displaced radially from the principal point. Let (x_n, y_n) be ideal normalized coordinates and $r^2 = x_n^2 + y_n^2$. The distorted normalized coordinates are:

$$x_d = x_n (1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \dots), \tag{29}$$

$$y_d = y_n (1 + k_1 r^2 + k_2 r^4 + k_3 r^6 + \dots), \tag{30}$$

where k_1, k_2, k_3 are **radial distortion coefficients** estimated during calibration.

- **Barrel distortion** ($k_1 < 0$): image bulges outward; straight lines bow away from center. Common in wide-angle lenses.
- **Pincushion distortion** ($k_1 > 0$): image pinches inward; straight lines bow toward center. Common in telephoto lenses.

The distorted reprojection error adds distortion correction to the pipeline: $(x_n, y_n) \rightarrow (x_d, y_d) \rightarrow K \rightarrow (u, v)$.

2.12 Calibration Methods

2.12.1 Reference-object (3-D target) calibration

Place an object of precisely known 3-D geometry in the scene (e.g., a calibration rig with marked corner points), identify corresponding 2-D image points, and solve via DLT.

- **Pros**: Closed-form solution; straightforward to implement.
- **Cons**: Requires a precisely manufactured 3-D object; does not model radial distortion without extension; cannot impose known constraints (e.g., known aspect ratio, zero skew).

2.12.2 Multi-plane (Zhang’s) calibration

Photograph a flat planar pattern (e.g., checkerboard) from at least 3 different viewpoints. Corner positions on the plane are known in 2-D pattern coordinates; their 3-D positions are inferred from the known planar structure. Each viewpoint provides a **homography** (a 3×3 projective transform between the plane and the image) from which constraints on K are extracted.

- **Pros**: Only requires a printed checkerboard; positions and orientations of the board need not be known; handles radial distortion; implemented in MATLAB (Bouguet toolbox) and OpenCV.
- **Cons**: Requires at least 3 views; involves nonlinear optimization for final refinement.

2.13 Augmented Reality Application

The full camera pipeline is illustrated concretely by a simple AR system:

1. **Detect** known 2-D feature points on a flat AR marker (e.g., a QR-code border or checkerboard).
2. **Estimate pose** $[\mathbf{R} \mid \mathbf{t}]$ from the 2-D↔3-D correspondences using DLT followed by Levenberg–Marquardt refinement on reprojection error. K is pre-calibrated.
3. **Render virtual content** by projecting 3-D virtual geometry through the estimated $P = K[\mathbf{R} \mid \mathbf{t}]$:

$$\mathbf{x}_{\text{virtual}} \sim P \mathbf{X}_{3\text{D}}.$$

4. **Composite** the rendered virtual image onto the live camera feed, producing the AR output.

The quality of the overlay depends directly on the accuracy of K (calibration) and $[\mathbf{R} \mid \mathbf{t}]$ (pose estimation).